PURSUIT

Prospect Search & Intelligence Engine

Implementation Plan — Apollo Integration + Service Architecture + Claude Code

Implementation constraint: No AI/LLM model will be used in the initial release. The intelligence layer will be implemented with deterministic rules, configurable scoring, signal processing, evidence validation, and traditional text matching where useful.

Source basis: Strategy Proposal v1.0 / PURSUIT-Analysis.md functional direction. The source recommends a two-stage, evidence-first prospect pipeline, separate scoring dimensions, confidence, entity resolution, signal freshness, explainability, a cold pool, and feedback. This plan translates those ideas into a non-LLM implementation.

Document	Pursuit Prospect Search — Engineering Implementation Plan
Version	1.0 — Initial implementation blueprint
Target	MVP suitable for phased delivery; first production-ready slice can be delivered in ~1 week with focused scope.
Primary integration	Apollo data source through an isolated Apollo client/service adapter
AI/LLM	Not used in initial release

Prepared for implementation with Claude Code

1. Executive Implementation Summary

The supplied strategy identifies the central problem as a search pipeline that is filter-bound, compresses prospect quality into one score, lacks evidence/freshness/entity resolution, and permanently discards low scores. The implementation should therefore prioritize the data and pipeline foundations before any AI layer. filecite turn0file0 L14-L38

Recommended implementation principle

- Build a modular monolith first: one backend service, one database, one worker/scheduler, and one frontend. Keep provider integrations behind adapters so Apollo can later be supplemented by other sources.
- Make all scoring deterministic and versioned. Every score must be reproducible from stored criteria, signals, evidence, and weight versions.
- Treat missing data as UNKNOWN rather than FAIL. Preserve rejected candidates in a recoverable cold pool.
- Do not let Apollo-specific fields leak into the domain model. Normalize Apollo responses into an internal Candidate/Company model.
- Do not generate prose with an LLM. The first release should assemble explainable prospect briefs from structured facts, templates, evidence, and deterministic rules.

Scope decision

Phase 1 must land first: criteria object, Apollo discovery, entity resolution, suppression, hard filters, rejection reasons, and cold pool. The source explicitly recommends Phase 1 before expanding the intelligence layer. filecite turn0file0 L485-L500

Document map

Section	Purpose
2	Technology stack and engineering choices
3	Target architecture and service boundaries
4	Core domain/data model
5	Apollo client/service design
6	Two-stage search implementation
7	Deterministic scoring, signals, evidence and ranking
8	API and frontend contract
9	Background jobs and scheduling
10	Testing, observability and security
11	One-week execution plan
12	Claude Code implementation workflow and prompts
13	Acceptance criteria and Definition of Done
14	Future LLM/AI extension points — explicitly deferred

2. Technology Stack

The goal is to use a stack that is fast to implement with Claude Code, easy to test, and does not require an AI/LLM dependency.

Layer	Recommended technology	Reason / role
Frontend	React + TypeScript + Vite	Prospect search UI, results, score breakdown, evidence, feedback. Use the existing frontend conventions if the project already has a React application.
Backend API	Node.js + TypeScript + Fastify or Express	REST API, validation, orchestration. Fastify is preferred for a lean service.
Apollo integration	Typed ApolloClient module + ApolloService adapter	Centralizes Apollo authentication, pagination, retries, rate-limit handling, normalization and provider DTOs.
Database	PostgreSQL	Relational model fits companies, criteria versions, evidence, signals, scores, feedback and audit history.
ORM	Prisma	Type-safe database access and migrations; aligns well with TypeScript/Node development.
Validation	Zod	Runtime validation of API input, provider responses and internal contracts.
Jobs	Node worker + cron / BullMQ later	Nightly discovery, signal refresh and cold-pool re-scoring. Start with a simple scheduled worker; add a queue when volume requires it.
Cache / rate limiting	Redis (optional MVP)	Useful when Apollo calls become frequent or concurrent. Do not make Redis mandatory for the first vertical slice.
Testing	Vitest/Jest + Supertest + Playwright	Unit tests for scoring; integration tests for API/database; E2E for the main search-to-result flow.
API docs	OpenAPI/Swagger	Makes the service contract explicit and easier for frontend and future integrations.
Observability	Structured logs + request IDs + metrics	Track provider failures, search counts, scoring distribution, job duration and feedback outcomes.
Deployment	Docker + existing cloud/CI pipeline	Repeatable local/CI/prod builds. Avoid introducing Kubernetes unless the existing environment requires it.

Important clarification: Apollo integration is treated here as a provider API/client service, not Apollo GraphQL. The architecture isolates it so the exact Apollo endpoint/version can be swapped without changing the domain layer.

3. Target Architecture

The source architecture is a two-stage pipeline: recall-first candidate discovery followed by precision-focused qualification, with feedback returning to criteria, discovery and scoring. filecite turn0file0 L428-L484

Recommended MVP architecture

Frontend (React)
Search Criteria → Search Job → Candidate List → Prospect Detail → Evidence/Score → Feedback

↓ REST/JSON

Backend API
Criteria Service · Search Service · Candidate Service · Scoring Service · Evidence Service · Feedback Service

↓

Provider / Infrastructure Adapters
ApolloClient → ApolloService → Normalized Company/Candidate data
Optional Web/News/Jobs adapters → normalized signals

↓

PostgreSQL
Criteria versions · companies · candidates · sources · evidence · signals · scores · feedback · audit events

Worker/Scheduler
nightly Apollo discovery → initial filtering → qualification → scoring → ranking → queue update

Service boundaries

Module	Responsibility	Must not do
CriteriaService	Create/version criteria; separate must-have, nice-to-have, exclusions and intent.	Call Apollo directly or calculate scores.
ApolloService	Authenticate, search, paginate, retry, rate-limit, normalize Apollo data.	Apply business scoring rules.
DiscoveryService	Execute discovery strategies and merge candidates.	Own provider-specific HTTP code.
EntityResolutionService	Resolve domain/name/country duplicates and merge source references.	Delete duplicate history.
FilteringService	Suppression + hard must-have rules + reachability sanity checks.	Use expensive research/scoring to decide basic eligibility.
SignalService	Store typed, dated positive/negative signals and apply decay.	Invent signals without evidence.
EvidenceService	Store source, date, claim, directness, corroboration and reliability.	Hide conflicting sources.
ScoringService	Calculate dimension scores, penalties, confidence and final score.	Depend on UI or provider DTOs.
RankingService	Rank by score, confidence, opportunity and timing; enforce diversity.	Change the underlying score.
FeedbackService	Capture relevant/not relevant, meeting, opportunity, won/lost and reasons.	Silently rewrite past scores.

Repository structure

```
src/
├── modules/
│   ├── criteria/
│   ├── discovery/
│   ├── apollo/
```

```
│ ├── companies/
│ ├── filtering/
│ ├── signals/
│ ├── evidence/
│ ├── scoring/
│ ├── ranking/
│ ├── feedback/
│ └── jobs/
├── shared/
│ ├── db/
│ ├── http/
│ ├── validation/
│ ├── logging/
│ └── errors/
└── app.ts
```

4. Core Data Model

The source requires versioned criteria, one resolved company per entity, dated evidence, typed signals, score dimensions, confidence, rejection reasons and feedback history. 【filecite】turn0file0】L78-L92】 【filecite】turn0file0】L109-L126】

Table	Key fields	Purpose
criteria_versions	id, name, version, must_have_json, nice_to_have_json, exclusions_json, intent_json, weights_json, active	Reproducible definition of an ICP and its scoring weights.
companies	id, domain, normalized_name, legal_name, country, industry, size, website, status	Canonical resolved company entity.
company_sources	company_id, provider, external_id, raw_payload, first_seen_at, last_seen_at	Preserve Apollo and future provider references.
search_runs	id, criteria_version_id, started_at, completed_at, status, counts_json	Audit each discovery/qualification execution.
candidates	id, company_id, discovery_strategy, state, rejection_reason, last_evaluated_at	Candidate lifecycle including cold pool.
signals	id, company_id, type, polarity, observed_at, source_id, relevance, decay_class	Positive/negative intent and need signals.
sources	id, type, url, title, publisher, published_at, fetched_at, reliability	Evidence source registry.
evidence	id, company_id, source_id, claim_type, claim_text, observed_at, directness, corroboration, status	Dated, attributed facts behind scoring.
prospect_scores	id, company_id, criteria_version_id, raw_score, penalty, confidence, final_score, band	Immutable/versioned score snapshot.
score_dimensions	score_id, dimension, value, weight, evidence_ids_json, rationale	ICP Fit, Business Fit, Need, Intent, Timing, Opportunity, Evidence.
feedback	company_id, user_id, event, reason, created_at, metadata	Learning signal from rep outcomes.
audit_events	actor, action, entity, entity_id, before_json, after_json, created_at	Traceability for operational and scoring changes.

Suggested candidate states

DISCOVERED → FILTERED → QUALIFICATION_PENDING → QUALIFIED → OUTREACH → NURTURE → COLD → SUPPRESSED. Rejected/cold candidates remain stored and can return to qualification when new signals arrive. The source explicitly replaces permanent discard with a recoverable cold pool. 【filecite】turn0file0】L231-L238】

5. Apollo Client & Service Design

Apollo should be an infrastructure dependency, not the business-logic layer. The application should consume a normalized internal contract.

Recommended flow

```
SearchService → DiscoveryStrategy(ApolloFirmographicStrategy) → ApolloService → ApolloClient → Apollo API → Apollo DTO → Normalizer → CompanyCandidate
```

ApolloClient responsibilities

- Store base URL and API key configuration outside source control.
- Expose only low-level provider operations: search, retrieve company/person, pagination and provider-specific metadata.
- Implement request timeout, retry for transient failures, rate-limit handling and structured error mapping.
- Log request IDs and provider status, but never log API keys or sensitive payloads.
- Return typed DTOs validated with Zod.

ApolloService responsibilities

- Translate internal DiscoveryQuery into Apollo-specific filters.
- Handle pagination and page-size limits.
- Normalize provider data into the internal CompanyCandidate shape.
- Deduplicate provider results before returning them.
- Attach provider/external IDs so entity resolution can merge multiple sources.
- Never calculate ICP score inside ApolloService.

Internal normalized contract

```
type CompanyCandidate = {
providerRefs: { provider: string; externalId: string }[];
name: string;
domain?: string;
country?: string;
industry?: string;
employeeCount?: number;
website?: string;
technologies?: string[];
locations?: string[];
contacts?: ContactCandidate[];
rawSourceRef: string;
};
```

Apollo search strategies for MVP

Strategy	Initial implementation	Priority
Firmographic	Industry, geography, employee count, company attributes.	
Signal-first	Start from known Apollo-accessible signals/attributes where available; otherwise create an adapter interface for future sources.	P1
Semantic/description	Traditional text matching: normalized keyword groups, synonyms, TF-IDF/cosine similarity if required. No LLM.	P1
Look-alike	Deterministic similarity against winning-company attributes; no learned model required for MVP.	P2
Relationship	Model subsidiaries/partners/portfolio/competitor relationships when source data is available.	P2
Inbound	Adapter for website-visitor or existing engagement data if already available.	P2

6. Two-Stage Search Implementation

The supplied strategy explicitly recommends Stage 1 for recall and Stage 2 for precision. filecite turn0file0 L373-L391

Stage 1 — Candidate Discovery / Recall

1. Load the active criteria version.
2. Run Apollo firmographic discovery.
3. Run additional configured discovery strategies where data is available.
4. Normalize all provider results.
5. Resolve entities by domain first; then normalized name + country.
6. Merge source references and discovery strategies.
7. Apply suppression rules.
8. Apply only hard must-have criteria.
9. Run reachability sanity check.
10. Store survivors for Stage 2; store rejected candidates with explicit reasons in the cold pool.

Stage 2 — Qualification / Precision

1. Collect available website/product, jobs, funding/news, technology and registry facts through provider/source adapters.
2. Convert each source into dated evidence records.
3. Detect positive and negative signals using deterministic rules.
4. Validate evidence: source reliability, date, directness and corroboration.
5. Calculate the seven raw dimensions defined by the source: ICP Fit, Business Fit, Need, Intent, Timing, Opportunity and Evidence.
6. Apply negative-signal penalties and confidence multiplier.
7. Assign score band.
8. Rank by business value/probability, freshness and decision-maker reachability.
9. Generate a structured, explainable prospect result from stored facts — not LLM prose.

Non-LLM qualification approach

Intelligence need	Non-LLM implementation
Criteria understanding	Structured UI/form + JSON schema. User explicitly selects must-have, nice-to-have, exclusions and intent. No natural-language interpretation required for MVP.
Semantic discovery	Keyword groups + synonyms + TF-IDF/cosine similarity over company descriptions. Keep this behind a SemanticStrategy interface.
Need detection	Rule catalog: hiring patterns, technology/end-of-life terms, expansion announcements, compliance/challenge terms, growth commitments.
Intent detection	Typed signal rules with dates and relevance weights.
Evidence validation	Source reliability score + freshness + directness + corroboration rules.
Explanation	Template-based rendering from score dimensions, signals, evidence and risks.
Learning	Outcome-based analytics and manual weight tuning/versioning; no model training in MVP.

7. Deterministic Scoring, Signals, Evidence & Ranking

The source proposes weights of 20/15/20/20/10/10/5 for ICP Fit, Business Fit, Need, Intent, Timing, Opportunity and Evidence, with confidence as a multiplier. filecite turn0file0 L209-L239

Initial score configuration

Dimension	Weight	Example deterministic inputs
ICP Fit	20	Industry, geography, size, language, mandatory profile criteria.
Business Fit	15	Business model, maturity, technology maturity, outsourcing/buying indicators.
Need	20	Hiring gap, legacy technology, compliance pressure, growth/capacity gap.
Intent	20	Recent hiring, funding, expansion, launch, migration, leadership change, strategic initiative.
Timing	10	Signal freshness and likely action window.
Opportunity	10	Estimated deal size/category, fit to service offering, reachable decision-maker.
Evidence	5	Source quality, freshness, directness, corroboration.

Formula

Raw Score = Σ(dimension score 0–1 × dimension weight)
Final Score = (Raw Score − Negative Signal Penalty) × Confidence Factor
Confidence ranges from 0.5 for thin evidence to 1.0 for well-corroborated evidence. The source uses confidence as a multiplier so weak evidence cannot be compensated for by a high raw score. filecite turn0file0 L222-L230

Signal freshness / decay

- 0–90 days: full weight.
- 91–180 days: reduced weight.
- 181–365 days: low weight.
- >12 months: near-zero unless structural.
- Structural signals must be explicitly tagged as structural so they do not decay like transient intent.

Recommended deterministic confidence calculation

Start simple and transparent. Example: Confidence = clamp(0.5 + source_quality_bonus + freshness_bonus + corroboration_bonus + directness_bonus, 0.5, 1.0). Store the component values so the result is explainable. Do not optimize these values prematurely; tune them from outcome data.

Bands

Final score	Action
≥ 70	Outreach queue — enrich contacts and make prospect visible to reps.
40–69	Nurture pool — monitor for new signals.
< 40	Cold pool — retain and re-score when new signals arrive; never permanently discard.

8. API & Frontend Contract

The UI should make the evidence and score dimensions visible, because the source defines an explainable prospect result as a required output rather than a score alone. filecite turn0file0 L392-L406

Core REST endpoints

Method	Endpoint	Purpose
POST	/api/criteria	Create a new criteria version.
GET	/api/criteria/active	Return active criteria.
POST	/api/search-runs	Start a discovery/qualification run.
GET	/api/search-runs/:id	Run status and counts.
GET	/api/prospects	Paginated ranked prospects with filters.
GET	/api/prospects/:id	Full explainable prospect detail.
GET	/api/prospects/:id/evidence	Evidence and sources.
POST	/api/prospects/:id/feedback	Capture rep feedback/outcome.
POST	/api/prospects/:id/research	Re-run qualification for one company.
POST	/api/jobs/nightly-search	Internal/admin trigger for scheduled run.

Frontend screens

- Criteria Configuration — mandatory criteria, nice-to-haves, exclusions, intent definitions and weights.
- Search Run Dashboard — status, candidates discovered, filtered, qualified, outreach, nurture, cold, errors.
- Prospect Queue — ranked list with score, confidence, top trigger, risk, source count and last signal date.
- Prospect Detail — company profile, score dimensions, signals timeline, evidence, risks, opportunity, source links.
- Feedback panel — relevant/not relevant reason, contacted, meeting, opportunity, won/lost and reason.
- Admin/Operations — provider health, failed jobs, stale data and re-run controls.

Explainable prospect response shape

```
{
company: {...},
overall: { finalScore, band, confidence, criteriaVersion },
dimensions: [{ name, score, weight, evidenceIds, rationale }],
whyMatch: [...],
need: [...],
intentSignals: [...],
evidence: [...],
risks: [...],
opportunity: {...},
lastEvaluatedAt
}
```

9. Background Jobs & Scheduling

The existing concept includes a nightly 2 AM pull. The source also expects stale information to be re-evaluated and cold-pool candidates to be re-scored when new signals arrive. filecite turn0file0 L9-L13 filecite turn0file0 L279-L304

Job	Frequency	Flow
Nightly discovery	Daily, e.g. 2 AM	Load criteria → Apollo discovery → normalize → resolve → suppress → hard filter → persist.
Qualification	After discovery / queued	Research adapters → evidence → signals → scoring → ranking.
Signal refresh	Daily or source-specific	Fetch new/changed signals and attach to companies.
Cold-pool re-score	Daily/weekly	Only re-evaluate companies with new material signals or periodic audit sample.
Stale-account refresh	Weekly	Find evidence older than threshold and queue for re-research.
Feedback aggregation	Daily	Compute score-band/outcome metrics and signal performance.

Idempotency requirements

- Every search run has a unique run ID.
- Provider external IDs and canonical company domains are unique where possible.
- Repeated jobs update existing evidence/signals rather than blindly inserting duplicates.
- Score snapshots are versioned; re-running a score creates a new evaluation rather than mutating historical results.
- Jobs must be safe to retry after partial failure.

10. Testing, Observability & Security

Testing strategy

Level	Tests	Examples
Unit	Pure business logic	Hard filters, normalization, entity matching, signal decay, confidence, score calculation, ranking.
Integration	Database + services + mocked provider	Apollo pagination, persistence, dedupe, search run lifecycle.
Contract	API/provider schemas	Zod DTO validation; API response contract.
E2E	Frontend → API → test DB	Configure criteria → run search → inspect prospect → submit feedback.
Regression	Golden test cases	Known prospect scenarios from source: perfect ICP/no need, strong need/poor ICP, old info, conflicting info, duplicate company, high score/low confidence.

Required observability

- Correlation/request ID across API → job → Apollo call.
- Structured logs for search run start/end, provider latency/errors, candidate counts and scoring outcomes.
- Metrics: Apollo success/error/rate-limit counts; discovery yield; dedupe rate; hard-filter rejection rate; qualification rate; score-band distribution; stale evidence count; cold-pool reactivation rate; feedback outcomes.
- Never log API secrets. Redact sensitive contact information where not needed.

Security

- Apollo credentials in environment secrets/secret manager, never committed.
- Role-based access for criteria changes, manual re-runs and feedback administration.
- Validate every external/provider payload before persistence.
- Use parameterized ORM queries and standard HTTP security headers.
- Maintain audit records for criteria/weight changes and manual prospect decisions.

11. One-Week Implementation Plan

A one-week delivery is realistic for an MVP if the scope is kept to Apollo discovery + filtering + deterministic scoring + explainable results, and if deeper external research sources are implemented through stubs/adapters first. The source itself recommends incremental delivery beginning with filtering/entity resolution. □filecite□turn0file0□L485-L510□

Day	Deliverables	Exit condition
Day 1	Repo setup; TypeScript backend; PostgreSQL/Prisma; config; logging; base API; criteria schema; Apollo client skeleton; Docker/local run.	App boots; DB migrates; health endpoint works; Apollo credentials load from env; CI runs tests.
Day 2	ApolloService; Apollo search; pagination; normalization; company persistence; entity resolution.	Real or sandbox Apollo response becomes canonical company records with duplicate merging.
Day 3	CriteriaService; suppression; hard filters; rejection reasons; candidate lifecycle; cold pool.	Search run produces survivors + recoverable rejected candidates with reasons.
Day 4	Signals/evidence model; deterministic signal rules; freshness/decay; initial research adapter interfaces; evidence validation.	A candidate can have dated evidence/signals and validated/stale/conflicting states.
Day 5	ScoringService; weights; penalties; confidence; score bands; ranking; explainable result DTO.	Golden scenario tests pass and every score points to its evidence/criteria version.
Day 6	React queue + detail page; score breakdown; evidence; risks; feedback; API integration.	User can run/search, inspect ranked prospects and submit feedback.
Day 7	Nightly job; retries; E2E tests; observability; security review; seed/demo data; deployment; documentation.	End-to-end demo works and deployment is repeatable.

If time is tight

Do not attempt full semantic discovery, web crawling, all external research providers, advanced look-alike modeling, or automated weight learning in week 1. Build clean interfaces for them and ship the deterministic core first. This preserves the architecture without making the MVP dependent on unavailable data or AI models.

12. Claude Code Implementation Workflow

Use Claude Code as the implementation agent, but keep architectural decisions and acceptance criteria explicit. Give it one bounded task at a time and require tests before moving to the next module.

Recommended Claude Code operating loop

1. Start with repository inspection. Ask Claude Code to map the existing project, identify the current framework, package manager, scripts, database conventions and deployment setup. Do not let it replace working infrastructure unnecessarily.
2. Create an IMPLEMENTATION_PLAN.md containing the architecture, domain model, module boundaries and acceptance criteria from this PDF.
3. Implement one vertical slice at a time: criteria → Apollo discovery → entity resolution → filtering → scoring → API → UI.
4. After each slice: run typecheck, lint, unit tests, integration tests and a manual smoke test.
5. Ask Claude Code to show changed files and explain why each changed file is necessary. Avoid broad refactors.
6. Commit after each stable slice with a focused commit message.

Prompt 1 — Repository analysis

You are implementing the Pursuit Prospect Search Engine. First inspect the existing repository without changing code. Identify frontend/backend structure, package manager, TypeScript setup, database/ORM, environment configuration, test framework, Docker/CI, and existing API conventions. Compare them to the attached implementation plan. Produce a gap analysis and recommend the smallest set of changes needed. Do not introduce an AI/LLM dependency. Do not rewrite existing working infrastructure.

Prompt 2 — Foundation

Implement the project foundation from the plan: TypeScript service modules, environment validation with Zod, structured logging, PostgreSQL/Prisma schema and migrations, health endpoint, error handling, and test setup. Keep provider integrations behind interfaces. Add tests and run typecheck/lint/test. Do not implement scoring yet.

Prompt 3 — Apollo

Implement ApolloClient and ApolloService as isolated infrastructure modules. Add typed request/response DTOs, Zod validation, authentication from environment variables, pagination, timeout, retry for transient errors, rate-limit handling, normalized CompanyCandidate output, and unit tests using mocked Apollo responses. Business scoring must not appear in the Apollo module.

Prompt 4 — Filtering

Implement CriteriaService, EntityResolutionService and FilteringService according to the plan. Must-have criteria are hard filters; nice-to-haves never disqualify; missing data is UNKNOWN; suppression rules are explicit; duplicates are merged by domain then normalized name + country; every rejection is retained with a reason in the cold pool. Add deterministic unit and integration tests for all edge cases.

Prompt 5 — Scoring

Implement deterministic Prospect Scoring. Dimensions: ICP Fit 20, Business Fit 15, Need 20, Intent 20, Timing 10, Opportunity 10, Evidence 5. Raw score is the weighted sum. Final score = (Raw Score - Negative Signal Penalty) * Confidence Factor. Confidence must be between 0.5 and 1.0. Store criteria version, evidence references and component values. Add golden tests for all scenarios in the plan.

Prompt 6 — Explainable results

Implement the explainable prospect result without using an LLM. Build the result entirely from structured company data, score dimensions, evidence, signals, risks and deterministic templates. Every material claim must reference a dated source. Conflicting evidence must remain visible. Add API endpoint GET /api/prospects/:id.

Prompt 7 — Frontend

Implement the React prospect queue and detail view. Show final score, score band, confidence, criteria version, why it matches, need, intent signals with dates, supporting evidence/source links, opportunity, risks, and feedback actions. Do not generate free-form AI text. Use API DTOs as the single source of truth. Add loading/error/empty states and tests.

Prompt 8 — Final review

Perform a production-readiness review against IMPLEMENTATION_PLAN.md. Check architecture boundaries, security, secrets, Apollo rate limits, idempotency, duplicate handling, cold-pool recovery, scoring reproducibility, evidence traceability, test coverage, logging and deployment. Do not make large refactors automatically. Report issues grouped as blocker/high/medium/low and fix only blockers/high issues after approval.

13. Acceptance Criteria & Definition of Done

Functional acceptance criteria

- A user can create/version an ICP criteria definition with must-have, nice-to-have and exclusion rules.
- Apollo discovery returns normalized candidate companies and preserves provider references.
- The same company from multiple discovery paths resolves to one canonical company.
- Existing customers/open deals/do-not-contact/active sequence records can be suppressed.
- Missing firmographic information does not automatically reject a candidate.
- Every rejected candidate has a stored reason and remains recoverable in the cold pool.
- A qualified prospect has separate dimension scores and a final score with confidence.
- Final score bands are ≥70 Outreach, 40–69 Nurture, <40 Cold.
- Score calculations are reproducible from criteria version + evidence + signal state.
- Signals have type, source and date; stale signals lose influence.
- Every score-affecting evidence item has a source and date.
- Conflicting evidence is surfaced rather than silently averaged.
- Prospect detail shows risks and confidence explanation.
- Rep feedback is stored and associated with the prospect.
- New material signals can move a cold/nurture prospect back toward qualification/outreach.

Technical Definition of Done

- TypeScript typecheck passes.
- Lint passes.
- Unit + integration tests pass.
- At least one end-to-end happy path passes.
- Database migrations are reproducible from an empty database.
- No API secrets are committed.
- Apollo failures do not crash the entire search run; failures are recorded and retried according to policy.
- Search jobs are idempotent/retry-safe.
- Logs contain enough context to trace a search run without exposing secrets.
- README documents local setup, environment variables, migrations, tests and deployment.

Golden scenarios to automate

Scenario	Expected behavior
Perfect ICP, no need	High fit but not top priority; nurture/watch.
Good ICP, active need	Need increases score and can qualify.
Strong intent, wrong mandatory geography	Hard reject; positive signal cannot override must-have.
Old information	Decay lowers signal contribution/confidence and triggers refresh.
Conflicting sources	Conflict remains visible; evidence confidence is reduced.
Duplicate Apollo records	One canonical company; all provider references retained.
High raw score, low evidence	Confidence multiplier prevents top ranking.
Cold prospect gets new signal	Candidate becomes eligible for re-score.
Multiple ICPs	Score per ICP; highest score owns routing.

14. Future AI/LLM Extension Points — Explicitly Deferred

The supplied strategy is written around AI-assisted research/scoring, but the current implementation decision is to ship without AI/LLM models. The architecture should therefore make AI an optional future adapter, not a prerequisite.

Future capability	Current non-LLM implementation	Future replacement point
Natural-language criteria understanding	Structured criteria form/JSON.	CriteriaInterpreter interface.
Semantic company discovery	Keyword/synonym + TF-IDF similarity.	SemanticDiscoveryStrategy interface.
Deep research synthesis	Structured evidence records + deterministic extraction rules.	ResearchSynthesizer interface.
AI scoring	Versioned deterministic scoring service.	ScoringStrategy interface; keep deterministic strategy as fallback.
AI account brief	Template-generated structured brief.	BriefGenerator interface.
Learning model	Outcome analytics + manual weight versioning.	ModelTrainer/RankingStrategy later.

Do not design the database around a specific LLM vendor. Store facts, evidence, signals, scores, criteria and outcomes. These are the durable assets; an LLM can be added later as a consumer/producer around them.

15. Key Risks & Mitigations

Risk	Impact	Mitigation
Apollo API limits / changes	High	Isolate provider adapter; retries/backoff; cache where useful; contract tests; monitor rate-limit responses.
Insufficient external evidence	High	Treat missing evidence as unknown; lower confidence; do not fabricate; queue for later research.
Rule-heavy scoring is initially imperfect	Medium	Version weights/rules and capture outcomes. Tune from evidence rather than assumptions.
One-week scope expansion	High	Ship P0 first: Apollo + entity resolution + filtering + scoring + explainability. Keep other providers as interfaces.
Duplicate companies	High	Domain-first entity resolution plus normalized name/country fallback.
Stale signals	Medium	Date every signal and apply decay.
False negatives	High	Cold pool + re-score on new signals + periodic cold-pool audit.
Untraceable recommendations	High	Evidence required for score-affecting claims; source/date on every claim.

The strategy specifically emphasizes that false negatives are silent and should be measured through periodic cold-pool sampling/re-scoring, while false positives can be surfaced through rep rejection reasons. filecite turn0file0 L351-L371

Final implementation recommendation

Build the deterministic foundation first. The most important engineering outcome of week 1 is not an AI model; it is a trustworthy prospect data pipeline where every candidate is resolved, filtered, scored, ranked and explainable from stored evidence. Once that foundation is stable, Apollo can be supplemented with additional sources and an optional AI layer without redesigning the core domain.

Source alignment

This plan directly operationalizes the supplied strategy's seven final recommendations: separate scoring dimensions, confidence as a multiplier, evidence for material claims, two-stage search, multiple discovery strategies, recoverable cold pool, and a feedback loop. filecite turn0file0 L511-L530